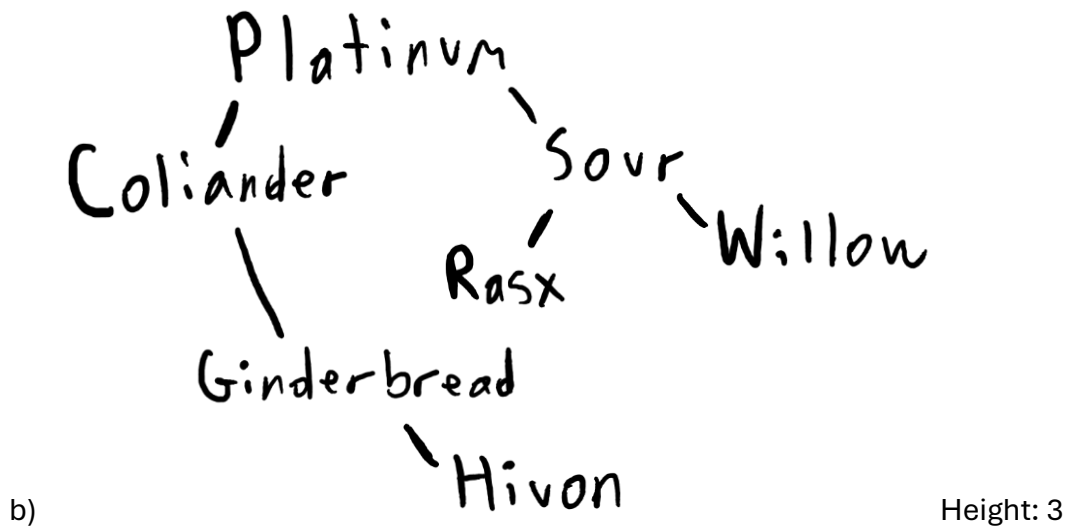
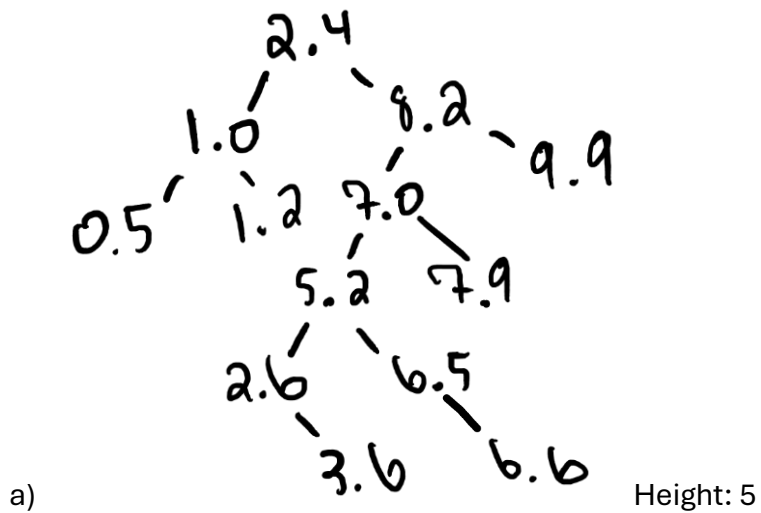
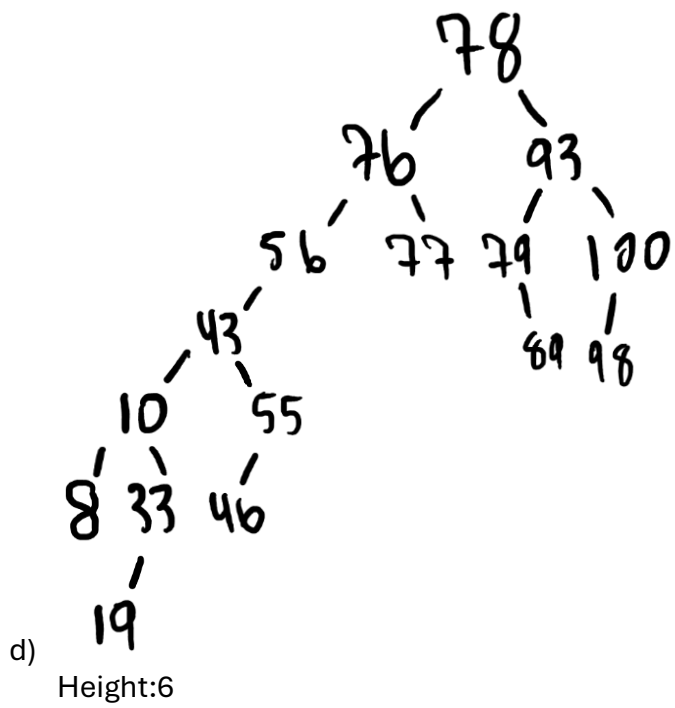
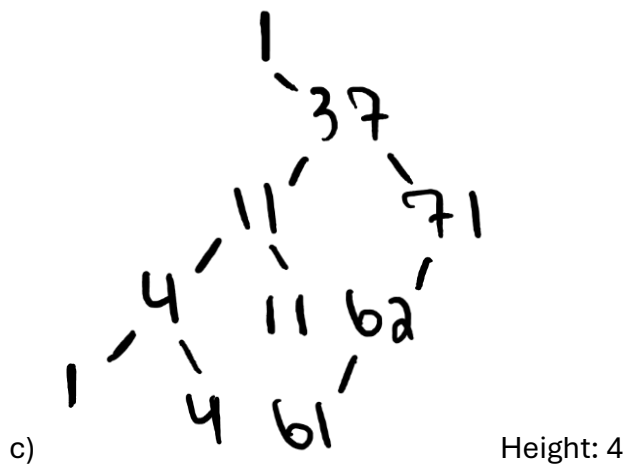


Problem 1:

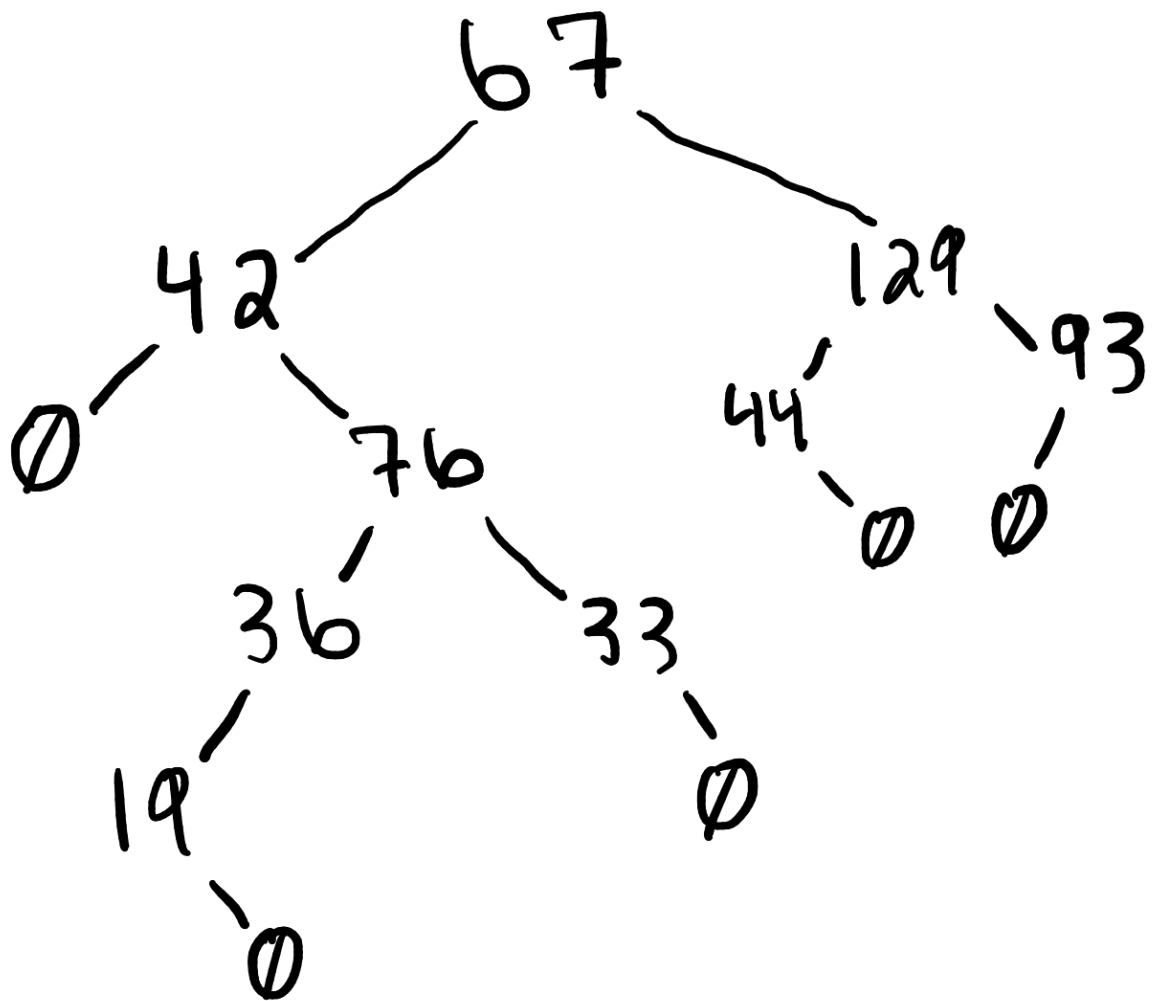
For this problem I used the BST rule that that child nodes on the left are less than the parent, while child nodes on the right are greater than or equal to the parent.



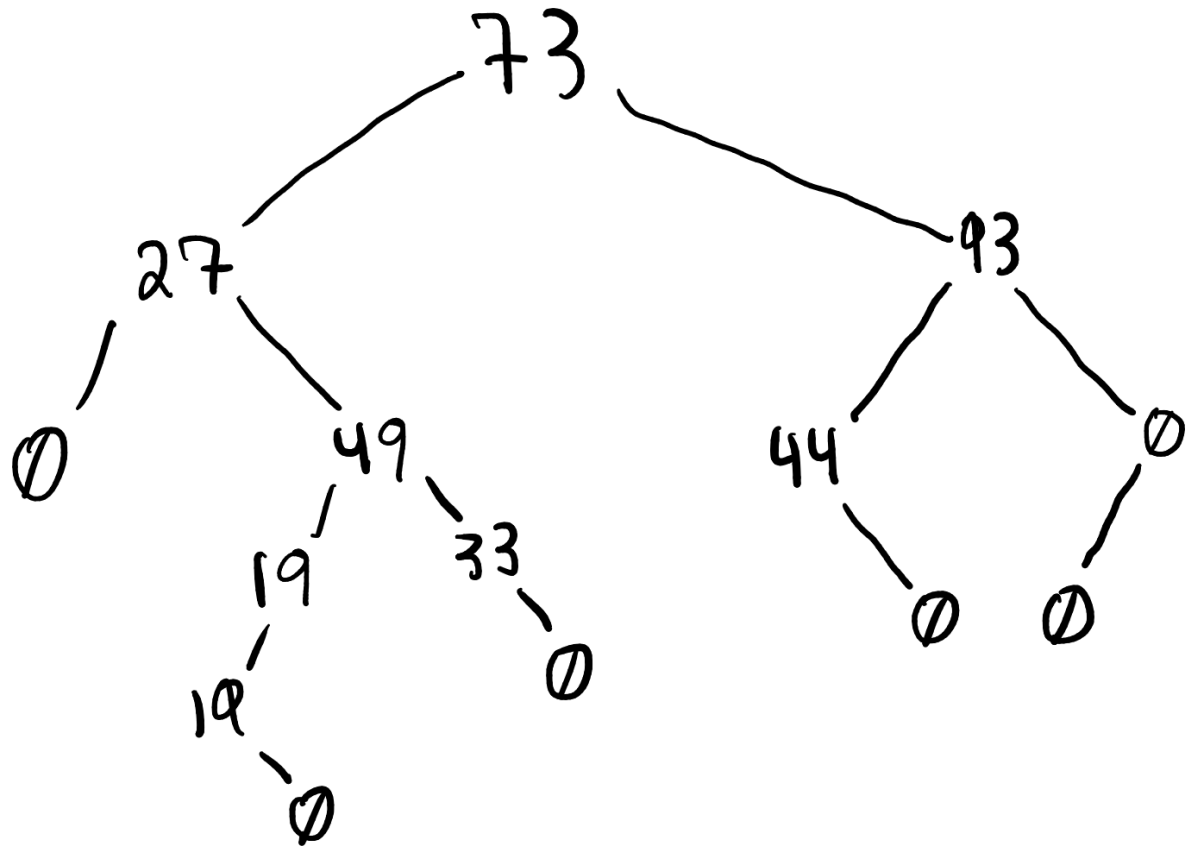


Problem 2:

a) After Preorder:



After Inorder:



- b) The resulting trees are not BSTs, this is because they do not meet the BST ordering property. In both of them all of the leaves are 0, which means that even on the right side of the root it's leaf is 0. Since 0 is less than the root, it should only be on its left side, but since it appears on its right side as the leaf, then in both cases it does not meet BST's ordering property, which means it is not a BST. For example in the preorder one 76 is the right child of 42 which is correct, but 42 is on the left of 67, which then means that 76 is also on the left of 67, but according to bst ordering this cannot be true, thus it is not a BST. Looking at that tree again, 76 has a left child of 36 which is the correct ordering but a right child of 33 which is less than it and thus break the ordering property. Then the inorder tree also breaks it, for example looking at node 49, it has a right child of 10 which is less than it and is on the left as it should be, but it's right child is 33, which is again less than it and thus breaks the ordering property. Furthermore, at node 93, 44 is on its left which is correct since it is less than it, but that is on the right subtree of 73, which means that it should be greater than 73, but $44 < 73$, thus it breaks bst ordering property. Therefore, both trees are not BSTs.

- c) The resulting trees are not AVLs, this is because they are not BST's and since to be an AVL it must be a BST, they are not an AVL. Even if they were, both don't change the tree's height or use rotations, so the shape of the tree is the same as the original just with different values. So, looking at the original tree, at node 21, its balance factor is $0 - (-3) = 3$. Since the balance factor of 21 is 3, its left and right sub tree heights difference by more than 1, which means that this tree violates AVL property. Since it violates the balancing property of an AVL, it is not an AVL tree. Since the original tree and the traversed one's are the same structure with different values, then the traversed ones are also not AVLs. Looking at the preorder tree at node 42. Its left subtree has a height of 0, while its right subtree has a height of 3, this means the balance factor for node 42 is $0 - 3 = -3$ and -3 is not -1, 1, or 0, which means it is not a valid balance factor and thus the preorder tree is not balanced by AVL standards and cannot be an AVL. Then looking at the inorder tree at node 27, its left subtree height is 0, while its right subtree height is 3. This means that the balance factor at node 27 is $0 - 3 = -3$, but since -3 is not -1, 0, or 1 it is not a valid balance factor. This means that node 27 is not balanced, which means the resulting tree after the inorder traversal is not an AVL either.

Problem 7:

Problem 3:

The worst-case time complexity for this algorithm is $O(n*m^2)$, where n is the length of the input string array and m is the length of the longest string in the input array. This is because the encodingSystem has 4 methods and the constructor for the class. The class constructor creates 3 variables, one is a new Huffman object, and two arrays that have a size equal to the length of the input so n . So far, that has a complexity of $O(1+n+n) = O(2n+1) = O(n)$ since constants and lower order terms are dropped. Then after those there is a for loop, which in its body calls on three Huffman methods. First it calls on frequencyCount for the current string. The frequencyCount method updates a variable in the Huffman object, then it has two loops sequentially. The first loop updates all values in the frequency array to 0, and its body just updates a slot in the array to be 0, which is $O(1)$. The loop itself runs from 0 to frequency length and does a comparison, method call to `.length`, and increment each iteration. The comparisons, and increment are constant time operations, and `.length`

just returns a variable so it is also constant time. Since the frequency array is always initialized to 27 no matter the input size, that means the loop will always run 27 times, which means that the complexity of the loop itself is $O(2 \cdot 27 + 2) = O(1)$ since constants are dropped. The complexity of a loop is the body times the loop, so it will be $O(1 \cdot 1) = O(1)$. Then after that loop there is another one, which runs from 0 to the length of the input text and increments and compares by one. The body of that loop just does an increment, array retrieval, method call to `charAt` and `charToNum`. `CharAt` just does an array like retrieval, which means that its is $O(1)$, and `charToNum` just does comparisons and returns, which are just $O(1)$. That means the body is just $O(1)$ overall. The for loop itself call on the `string.length()` method which is just a retrieval so $O(1)$. Since the number of times varies on the input string and our input has a list of string, then the most it will ever run is the length of the longest string, so in the worst case it will run that length every iteration, thus the loop will run m times. That means the loop has a complexity of $O(m)$. The loop as a whole will then have a complexity of $O(m \cdot 1) = O(m)$. That means the complexity of `frequencyCount` method is $O(m + 1 + 1) = O(m + 2) = O(m)$ since constants and lower order terms are dropped. Then after that call to frequency count, it calls on `buildHuffman()`. This method first initializes an array to be size 53 no matter the input size so its has a complexity of $O(53) = O(1)$ since constants are dropped. Then it initializes a variable to be 0. Following that is a for loop, which has an if statement in the body, which is taken in the worst case each iteration since it results in more operations being done. The body of if creates a new heap node, array retrievals, increments, and call on `numToChar()`. All of those are constant time operations, `numToChar()` just does a lot of comparisons and returns which are all just $O(1)$, so that means the body of this is $O(1)$. The if just does a comparison and array retrieval also $O(1)$. Then the loop itself runs from 0 to `frequency.length` which means it will always run 27 times. Thus the complexity of the loop is $O(1 \cdot 1) = O(1)$. Then there is an if statement that is not taken in the worst case otherwise it would return and result in less operations. After that are five variable declarations and initializations and one arithmetic, which are all constant time operations overall being $O(1)$. Following those is a for loop, which runs has body with only updates, increments, creating a new `HeapNode`, and a method call to `findSmallest`. The creation of a new heapnode just does some arithmetic and updates pointers which are all $O(1)$ operations as well as the updates and increments being $O(1)$. The complexity of `findSmallest` is $O(1)$, this is because it first initializes a variable once and then has a for loop, which runs from 0 to the largest input given to it, but in that `buildHuffman` method, the largest possible that can be given to it is 53 since it is first called on with a size of `frequency - 1` which is 26 and increases to the current size each time, but the current size is upperbounded by the array size of 53 since that is what is keeping track of. This means the for loop itself is $O(1)$. Then its body just has comparisons, array retrievals, and call to `getFrequency()`. The `getFrequency` method just returns a variable so its

a constant in time just like the other operations, This means the body is $O(1)$. Then that means the for loop is $O(1)$ as a whole. Then there is a return statement which is a constant time operation, which makes the complexity of findSmallest $O(1+1+1) = O(3) = O(1)$ since constants are dropped. This means the body of the for loop for the buildHuffman method is $O(1)$. The for loop runs from 0 to merges, and merges is equal to current size - 1, which was calculated before the loop was entered. Since the currentSize is updated in the previous loop that means it is upper bounded by the number of iterations of the previous loop, 27, which means merges is at most $27-1 = 26$, thus the loop will only run 26 times at most this having a complexity of $O(1)$ itself. That means the complexity of the loop is $O(1*1) = O(1)$. After that is an update, array retrieval and arithmetic, which are constant time operations thus being $O(1)$. After that is a method call to genCode(). The genCode() method has an if statement that is not taken since it would result in less operations being done, and it just as a comparison in it so it is just $O(1)$. Next, is a for loop which has a body that just does an update. The for loop itself runs from 0 to codes.length and does increments and comparisons, but the length of codes is always going to be 27 since that is what it is initialized to in the constructor. This means the loop itself has a complexity of $O(1)$, meaning the loop as a whole has a complexity of $O(1*1) = O(1)$. After that a variable is declared which is $O(1)$, and then there is another for loop. That loop's body has an if segment is taken each iteration in the worst case. The if segment has a condition with just a comparison and an array retrieval which is $O(1)$. Then its body has an update and method call to numToChar() which is constant so that line is $O(1)$. Following that is another update, but also a method call to getPath. That method does an preorder traversal of the array to find the path to the leaf node, which are the characters in a Huffman. In the worst case, the structure of the resulting tree would be skewed to one side similar to a linked list, but since the number of characters being analyzed is 27, hence the frequency and codes being 27, there will only be 26 nodes visits excluding the root. This method also has recursive call to move further down, so that means even if the input size is large, the most amount of time this can be called is 26 times. This method first has two if state notes which are not taken until the last recursive call since it would result in less operations happening. Then there is a string being initialized to the result of a recursive call to the left, but that recursive call also has a string concatenation. After that is another if statement which is not taken otherwise there would be less operations happening. Following that is a recursive call to the right which also has a string concatenation. There are also comparisons to see if the node is null or the target one, but those are just $O(1)$. So, that means the method, excluding the string concatenations are $O(1)$. The string concatenations will only be returned and occur on the correct path to that character's node, but since there are only 27 levels even in the worst case, and this can only be called 26 times in the worst case, that means the number of concatenations is limited to 26, and that is also the length of the longest possible code.

Since strings in java create a new one each concatenation equals to the current one plus the appending one, that means a string of 0, then 1, then 2, then 26 will be created. Even so, this is always going to be upper bounded by 27^2 since that concatenation follows the sum of positive integers, which is sum from $i=0$ to 26 of i , which is equal to $26(26+1)/2 = 351$, which means it is $O(351) = O(1)$ in the worst case since constants are dropped. Then that means the complexity of the gen code method's second for loop's body is $O(1+1+1) = 3$ $O(3) = O(1)$ since constants are dropped. Thus the second loop has a complexity of $O(1*1) = O(1)$. Then that means the complexity of genCode() is $O(1+1+1+1) = O(4) = O(1)$ since constants are dropped. Then that means the complexity of buildHuffman is $O(1+1+1+1+1+1+1) = O(7) = O(1)$ since constants are dropped. Then in the constructor the encode method is called, which first initializes a string which is $O(1)$. Then there is a for loop, with an if statement that is always taken in the worst case since it results in the most operations being done. The if statement does a array retrieval, method calls to charToNum, and charAt(), which are all constant time operations being $O(1)$. Then the body does a string concatenation to rebuild the code, in the worst case this is appending 26 characters to what ever the current result is, since 26 is the length of the longest code as explained previously. This means it creates a string of the previous plus 26. The loop itself runs from 0 to the length of the input text and it is upper bounded by m because the most this loop will ever run will depend on the length of the longest input string, since everything else will run the same or less times than it. The loop runs m times and does that concatenation each time, which means a new string is being made and the transfers happening to build the new string from the old one to the new one is $0*26 + 1*26 + 2*26 + \dots + m*26$, which if 26 is substitute out follows the sum of positive integers from $i=0$ to m of i . This equals $26 * (m(m+1))/2 = 26(m^2+m/2) = m^2$ since constants and lower order terms are dropped. This means the complexity of the for loop is $O(m^2)$ across all iterations. Following that is a return statement which is a constant time operation. This means the complexity of the encode method is $O(1+m^2+1) = O(m^2+2) = O(m^2)$ since lower order terms are dropped. This means the complexity of the for loop inside of the encoding system constructor is $O(1+1+m^2) = O(m^2+2) = O(m^2)$ since constants and lower order terms are dropped. The loop itself runs from 0 to the length of the input string, and does comparisons and increments each iteration and stops when the length is reach which is n iterations. So the complexity of the loop itself is $O(2n+2) = O(n)$ since lower order terms and constants are dropped. This means the complexity of the for loop is $O(n*m^2) = O(n*m^2)$. Following that constructor is the printCodes() method. This method only has a for loop. The loop's body just has a print statement which also does an array retrieval meaning it has a complexity of $O(1)$. Then the loop itself runs from 0 to encoded.length, which is initialized to be the same length as the input array, and do comparisons and increments meaning in it will run n times. This means it has a complexity of $O(2n+2) = O(n)$ since lower order terms and

constants are dropped. That complexity of the loop and there for the method itself is $O(n \cdot 1) = O(n)$. Following that method call is the `highestCode()` method, which starts off with an if statement with a comparison, but it is not taken since it would result in less operations as it would return early and it $O(1)$. Then it initializes a variable which is $O(1)$ and then has a for loop. The loop's body has an if statement, which has a comparison, array retrieval, and method call to `.length` for a string which are all $O(1)$ operations. The body is just an update which is $O(1)$, so the loop's body has a complexity of $O(1)$ even if the if is not taken. The loop itself runs from 0 to `encoded.length`, which this loop has previously been stated to have a complexity of $O(n)$. This the complexity of the for loop is $O(n \cdot 1) = O(n)$. Then there is a return statement with an array retrieval which is again a constant time operation of $O(1)$. Thus the `highestCode()` method has a complexity of $O(1+1+n+1) = O(n+3) = O(n)$ since lower order terms and constants are dropped. Following that is a method call to `shuffleCodes()`. This method first has an if condition that is not taken otherwise less operations would happen, but eh dint ion is a comparison and call to `.length` which are constant in time $O(1)$. Then there is a print statement which is $O(1)$ followed by the initialization of a new `Random` object which is also $O(1)$ and then an integer initialization of $O(1)$. Following those is a for loop, which has a body that updates a variable to be `.nextInt` of the `Random` object, which just does arithmetic and returns the value, so that mean it's complexity is $O(1)$. Then it initialized a variable, array retrieval, and then does updates, which are all $O(1)$ operations, meaning the body has a complexity of $O(1+1) = O(2) = O(1)$ since constants are dropped. The for loop itself runs from 0 to `encoded.length`, which was previously determined to mean that it has a complexity of $O(n)$. That means the loop a has a complexity of $O(n \cdot 1) = O(n)$. After that for loop is a return statement which is a constant time operation. This means the complexity of the `shuffleCodes()` method is $O(1+1+1+1+n+1) = O(n+5) = O(n)$ since constants and lower order terms are dropped. Following that is a method call to the last method `printCodeStates()`, which has a for loop identical to the construcor and a near identical body too. The body of that for loop first has two calls to `frequency count` and `buildHuffman`, which were determined to be $O(1)$ each. Then there is a call to `printStats()` for `Huffman`. That method fist has a print line which is $O(1)$, followed by a for loop, with a body with an if statement that is taken each iteration for more operations. Then there is another one inside of that which does not matter which one is taken since both paths do an array retrieval does print line which are all $O(1)$. The if stamens also have comparisons, and retrival, but they are all $O(1)$, which means the body of the for loop is $O(1)$ overall. The for loop runs from 0 to `frequency.length` which is determined previously to mean it has a complexity of $O(1)$, so the loop itself has a complexity of $O(1 \cdot 1) = O(1)$. Then there is another print line and a for loop but this is identical to the previous one, but instead it is running from 0 to `codes.length` which was determined to be $O(1)$ and does the array retrival from `codes`, but it does have the same

prints and comparisons, so that means the loop has a complexity of $O(1)$ overall. Then there is final print line, which call on the encode method, which we previously determined to be $O(m^2)$, which means the complexity of the printStats() method is $O(1+1+m^2) = O(m^2+2) = O(m^2)$ since constants and lower order terms are dropped. This means that the body for the printCodeStats method is $O(1+1+m^2) = O(m^2+2) = O(m^2)$ since constants and lower order terms are dropped. The for loop itself runs from 0 to inputs.length which we previously determined to be $O(n)$, which means the complexity of the loop and subsequently the method is $O(n*m^2)$. Thus the complexity of all of this as a whole is $O(n*m^2+n+n+n+n*m^2) = O(2n*m^2+4n) = O(n*m^2)$ since constants and lower order terms are dropped.

The worst-case space complexity for this algorithm is $O(n*m^2)$, where n is the length of the input array and m is the length of the longest string in the input array. This is because the constructor first creates two variables once a new Huffman and an array being equal to the input one. The first one has a complexity of $O(1)$ since it will happen once no matter the input size, but the second one's size will depend on the input size and be linear to it since it will create spaces to match it, making it $O(n)$. Then there is an array being initialized to have a size equal to the input size, which also makes it $O(n)$ since the space created does depend on the size of the input size and is linear to it. Following that is a for loop, which as previously mentions runs n times, but also only creates an integer i once no matter the input size making it $O(1)$. The loops body, then calls on frequency count, which first updates the string variable in the Huffman class to be the inputted one, which just updates the pointer to the value of the string in the input array. Following that is a for loop, which has a body that does updates and no deceleration meaning the body crest no space. The loop itself creates a variable i once no matter the input or text size, meaning its space complexity is $O(1)$ and thus the loop itself is $O(1)$ since its body doesn't also make any space. Following that is a second for loop which has a body that does array retrival, update, increment, and method calls to charToNum and char at. Both of those method do not create any space since they just to comparisons or returns, and the retrival updates and increments also just don't make any apace, meaning the body of the loop does not make any space. The loop itself makes an integer j once and no matter the input size so it and subsequently the loop has a space complexity of $O(1)$. That means the. Method has a complexity of $O(1+1+1) = O(3) = O(1)$ since constants are dropped. Then the constructor calls on the buildHuffman class, which first creates two variables an array of size 53 and an integer, and only does it once no matter the input size meaning it has an space complexity of $O(1)$. Following that is a for loop which creates an integer i once and it's body has an if statement that is always taken in the worst case. The body of the if does

an update, array retrieval, increments all of which don't make space, but it also creates a new HeapNode object, which does create space, and in the worst case this happens every iteration of the loop. The loop itself runs from 0 to frequency length which always 27 times, which means that it is constant since no matter the input size it will run 27 times, this also means that the number of nodes being created no matter how large input size grows will always be 27 in the worst case, thus making the for loop have a space complexity of $O(1)$.

The after that is a if segment that is not taken since it just has a return and the comparisons ones not make space. Followed by that are 5 variables being created but they are always created no matter the input size and only happen once, meaning their space complexity is $O(1)$. Then there is another for loop, which has a body with updates, increment, array retrieval which don't make space, but it also has a method call to findSmallest. That method creates two variables once no matter how large the input size is, since they are only made once. Then there is for loop after that is made and that loop only makes an integer l once no matter the input size, which just does array retrieval, comparisons, updates, and method call to get frequency, which just does a return without making space. So the complexity of the findSmallest method is $O(1+1)=O(2)=O(1)$ since constants are dropped. That means the space complexity of the for loop in the buildHuffman method is $O(1)$ since findSmallest does not make space. That for loop runs from 0 to merges, which is previously established to be upper bounded by 26 a constant number, which means in the worst case, the variables made by the findSmallest only happen 26 times, which means the space complexity of the loop is just $O(1)$ since constants are dropped. After that is an update, retrieval, and arithmetic which don't make space so its $O(1)$. Then there is a method call to genCode(), which starts with an if statement that is not taken since it would return early, and the condition also does not make space since its just a comparison. Then there is a for loop which has a body of just updates and only makes an integer l once in the for loop no matter the input, which means that first for loop has a complexity of $O(1)$. Then there is a integer getting made, but it only happens once regardless of input size then it has a complexity of $O(1)$. Then there is one last for loop, which has an if statement that is taken each iteration since it results in more space being made. That does updates and calls to numToChar which don't make space but it also calls on getPath method. That method has an if condition that is not taken since it results in less space being taken, and there is a second one that is also not taken since it does a comparison and a getData() call which don't make space. Then it creates a String variable and initializes it to getPath, and the creation itself is $O(1)$. The getPath method call does a string concatenation adding 0 or 1 to it. depending on it does the left or right recursive call. The getPath does a preorder traversal of the tree, but there are only 27 characters which mean there are 26 internal nodes it goes to before it exits on the leaf. This means that the concatenations can happen at most 26 times per character and thus makes 26 strings in the worst case. Since there are always at

most 26 strings being created in the worst case not matter the input size, that means the complexity of this method is $O(26) = O(1)$ since the amount of space does not increase as input size does. This means the complexity of the `genCode` method is $O(1+1+1(1)) = O(3) = O(1)$ as constants are dropped, since the amount of space made by `gen` does not increase as the input size does. This means the complexity of the `buildHuffman`'s space complexity is $O(1+1+1+1+1+1+1) = O(7) = O(1)$ since constants are dropped. Then in the `encodingSystem` constructor then calls on the `encode` method to update a value in the `encode` array. The `encode` method first creates a `st` ring, but only does it once no matter the size of the input array or length of the input array's strings. Then there is a `for` loop, which has an `if` statement that is taken each iteration since it results in the more space being created. The `if` condition does `n` array retrievals, comparisons and method calls to `charToNum` and `charAt()` which are constant in space since they don't make any variables. Then in it's body it does a string concatenation for `result` and the corresponding code of a character. Since the longest possible code for a character is 26 since that is the longest possible path `m`, that means each concatenation makes a new string with 26 characters and the previous results. So that means the space created by this loop is 0, then $1*26$, then $2*26$, then $3*26$ after the third iteration, so after `m` iterations it will be $m*26$. The reason it does `m` iterations is because the `for` loop runs from 0 to the length of the input text, since in the worst case will the length of the longest input string `m` and does depend on the input, since that is the most it will run for all strings in the input array. That means across all iterations it creates $0 + 1*26 + 2*26 + \dots + m*26$. space to move the previous one and the 26 characters, which if 26 is substituted out follows the sum of positive integers, which is the summation from `i=0` to `m` of `i`, which is equal to $O(26(m(m+1)/2)) = O(26(m^2+m)/2) = O(m^2)$ since constants and lower order terms are dropped. This means the space complexity of the `encode` method is $O(1+m^2) = O(m^2)$. This means the complexity of the `encodeSystem` constructor's `for` loop body would be $O(1+1+m^2) = O(m^2+2) = O(m^2)$ since constants and lower order terms are dropped. The `for` loop itself these three methods are called in runs from 0 to the `input.length`, which means it runs `n` time. Since it runs `n` time and makes m^2 space each time, the complexity of the `for` loop as a whole is $O(n*m^2)$. The `encodySystem` constructor has a space complexity of $O(1+n+n+n*m^2) = O(n*m^2 + 2n + 1) = O(n*m^2)$ since constants and lower order terms are dropped. Then there is a `printCode()` method that only has a `for` loop. The loop's body just does a `print` and `retrieval`, which don't make space and thus are constant in space. The `for` loop itself creates a integer `i`, but only doe sit once no matter how large the input size is. Since the loop doesn't create space in the body that means the space complexity of it and the method is just $O(1)$ since it only creates one space one. Following that is a method `highestCode()`, which first has an `if` segment that calls on `.length` for an array which doesn't make space and is not taken since it would result in less space being made. Then after that

it creates an integer variable but only once no matter how large the input or strings are. Following that is a for loop, but that loop has a body with only comparisons, updates, `.length()` calls, and array retrievals, which are all constant in space since they don't make any variables, which mean no matter how many times the loop runs it won't make any space in the body. The for loop itself declares a variable `i`, but only once no matter how large the inputs are meaning its space complexity of $O(1)$. Then there is a return and an array retrieval which are $O(1)$ since they don't make any new space. This means the complexity of the `highestCode()` method is $O(1+1+1) = O(3) = O(1)$ since constants are dropped meaning that the amount of space this method takes is always the same not matter the input size or text length. Then after that there is the `shuffleCodes()` method, which first has an if segment with a call to `.length` and a comparison, which don't make space and it isn't taken since it results in less space being made. Following that is a print statement, which are constant in time and space. After that is the creation of a new Random object, which will only happen once in the method no matter how large the input array or strings are, meaning it has a space complexity of $O(1)$. Then there is a for loop, which has a body that first does an update and calls on `nextInt` method and `.length`, which don't declare any variables thus making no space. Next the for loop creates a string variable `temp` and initializes it, since it does create a string variables it has a complexity of $O(1)$. Then after that there are just two updates and array retrievals which don't make space, thus being constant in space and $O(1)$. This means the body of the loop is $O(1+1+1) = O(3) = O(1)$ since constants are dropped. The for loop itself creates an integer `i` but only once no matter how large the input array or strings are thus being $O(1)$. It also does comparisons, call to `.length`, and increments, which don't make variables. The loop itself runs `n` times since it runs from 0 to `encoded.length`, and since each iteration will create a new string variable that means the loop itself makes $O(1+n)$ space from the loop itself and the body after all iterations, which is $O(n)$ since lower order terms are dropped. This means the complexity of `shuffleCodes` is $O(1+1+n+1) = O(n+3) = O(n)$ since constants and lower order terms are dropped. Finally, there is the `printCode` states method, which only has one for loop, that is identical to the one in the constructor with the only difference being it calls on the huffman's `printStats()` method instead of `encode`. The `printStats()` method, first has a print statement which is $O(1)$, then it as a for loop, which has a body of just comparisons, array retrievals, prints, and array retrievals, not creating space thus being $O(1)$. The loop itself creates a variable `i` once not matter or large the input array or strings are so its complexity is $O(1)$, which means the loop as a whole is just $O(1)$ since the body will never make any space regardless of the number of iterations. Then after that is a another print line which is $O(1)$. Then there is another for loop which has a body identical to the previous one but does retrievals on a different array. Since it still only has comparisons, retrievals, and prints, it has a complexity of $O(1)$ since it doesn't make any space. The for lop itself

also makes an integer i only once no matter the input size or string length, which gives it a complexity of $O(1)$. Since the loop's body never creates space that means no matter how many iteration it runs no space will be made thus the loop has a complexity of $O(1)$. After that is a print line and method call to encode, which was determined to have a space complexity of $O(m^2)$, meaning that line is just $O(m^2)$ since encode is the only thing that would make space. This means the complexity of the `printStats()` method is $O(1+1+1+1+m^2) = O(m^2 + 4) = O(m^2)$ since lower order terms and constants are dropped. Since we have previously determined the space complexity of the previous methods are $O(1)$, then that means the complexity of `printCodeStats()` for loop body is $O(1+1+m^2) = O(m^2+2) = O(m^2)$ since constants and lower order terms are dropped. The for loop itself creates a variable i but only once no matter the input or string size, and then it runs from 0 to `input.length` which means it runs n times. Since it runs n times and does the body each iteration the for loop would have a complexity of $O(1+n(m^2)) = O(1+n*m^2) = O(n*m^2)$. since lower order terms are dropped. This means the complexity of the `printCodeStats()` method is $O(n*m^2)$. So, the space complexity of the constructor and the methods as a whole is $O(n*m^2 + 1 + 1 + n+n*m^2) = O(2n*m^2 + 2+n) = O(n*m^2)$ since constants and lower order terms are dropped.

The best-case time complexity for this algorithm is $\Omega(n*m)$, where n is the length of the input array, and m is the length of the longest string in the input array. This is because in the best-case, since the strings will always be space or lower-case letters, this best-case input is an array of strings where each string only has one unique character. The `encodeSystem` constructor does first initialize three variables and call on `.length`, all of which are constant time operations. Two of those variables initialize and array to be the size of the input array, which means those have a $\Omega(n)$ complexity. After those is a for loop, which initializes a variables, does a comparison and increment until the incremented value reaches `text.length`, n , which means it runs n times. The loop itself then has a complexity of $\Omega(2n+2) = \Omega(n)$ since constants and lower order terms are dropped. The loop's body has three method calls with the first one being to `frequencyCount()`. That method first initializes a variable to be a string, which is constant $\Omega(1)$. After that is a for loop, which runs from 0 to `frequency.length`, and has a body that just does updates. The `.length` method is constants, as well as the comparisons and increments and initialization, but `frequency`'s length is always 27 which means the time complexity of the loop itself and its body are both $\Omega(1)$. This means the loop's complexity is $\Omega(1*1) = \Omega(1)$. Following that is another for loop, which has a body with just constant time operations of retrievals, increments updates. Call to `charToHum`, and `charAt`, which were previously mentioned to be $\Omega(1)$. This loop runs from 0 to the length of the input text, which is m since that is a most it would run even in the best case. This means the loop has a complexity of $\Omega(m*1) = \Omega(m)$.

This means the complexity of the frequency method is $\Omega(1+1+m) = \Omega(m+2) = \Omega(m)$ since constants and lower order terms are dropped. Following that there is a method call to `buildHuffman`, which first always makes an array of size 53 and initializes a variables so that is $\Omega(1)$, the rest of the method is the exact same as it was in the worst case scenario since even in the loops will still run 27 time at most and only perform constant time operations, which means this method up to the loops is $\Omega(1)$. In the loops there is a call to `findSmallest`, but as previously explain this will always have a constant time operation since it is independent of input size, is upper bounded by 53, and is made of constant time operations making it $\Omega(1)$. After the loops is an update and array retrieval which are $\Omega(1)$, but then there is a method call to `genCode()`, which is exactly the same as it was in the worst case because the number of operations perform in the method depends on the length of the frequency and code's array which are always 27, and thus constant. There is a method call to `getPath` in one of the loops though. The `getPath` method in the best case will have a time complexity of $\Omega(1)$. This is because it only has initializations, get methods, comparisons, concatenation, and recursive calls. All of them except the recursive calls and concatenations have a $\Omega(1)$ complexity. Since the best case only has one unique character that means there is only one recursive call and singular digit concatenation that will happen since one character means there is only one short path to the character node. This means the complexity of the `getPath` and subsequently the `genCode()` method is $O(1)$ since they only have constant time operations. This means the complexity of the `buildHuffman` is $\Omega(1)$ since it only has those operations and methods which are constant in time. After that the `encodeSystem` call on `encode` method, which like in the worst case which does the same as it did in the worst case running m times since it relies on the longest string length, but since it is only on character in the best case that means the codes are only 1 character, which means that update and concatenation is only appending one and is thus across all m iterations, it makes $2m$ characters, which means it has a complexity of $\Omega(2m) = \Omega(m)$ since constants are dropped. There is also an initialization and return but they are constant time operations, which means the complexity of this in the best case is $\Omega(1+1+m) = \Omega(2+m) = \Omega(m)$ since constants and lower order terms are dropped. This means the complexity of the `encodeSystem` constructor loop body is $\Omega(1+1+m) = \Omega(m+2) = \Omega(m)$ since lower order terms and constants are dropped. Since the for loop runs n times. that means the complexity of the loop is $\Omega(n*m)$, and thus the complexity of the constructor is $\Omega(1+n+n)$. Following that is a method call to `printCodes()`, and this method has a complexity that is the same as the worst case because of the same logic and. since, it will loop and do a constant time print and retrieval n times, thus being $\Omega(n)$. Following that, is a method call to `highestCode()`, which has the best-case time complexity of $\Omega(1)$ in the best-case since if the length of the input array is 0 it will immediately return, which means its complexity. is $\Omega(1)$. Then there is a method call to

shuffleCodes(), which has a complexity of $\Omega(1)$ because in the best case and the length of the input is null for this method, then it will immediately return which has a complexity of $\Omega(1)$. Finally, there is a method call to printCodeStats, which has a loop that is identical to the one in the constructor with the only difference being a call to printStats instead of encode. The complexity of printStats() is exactly the same as it was in the worst case up until the encode method call. This because even in the best case it will still print and go through two loops that will always run 27 times, meaning that they have an $\Omega(1)$ complexity. Then there is a print line and a call to encode, which was previously determined to in the best case be $\Omega(m)$. This means the complexity of the printStats() method is $\Omega(m+1+1) = \Omega(m+2) = \Omega(m)$ since constants and lower order terms are dropped. This means the body of the for loop for printCodeStats is $\Omega(1+1+m) = \Omega(m+2) = \Omega(m)$ since constants and lower order terms are dropped. Since the loop is like the one in the constructor and runs n times that means the complexity of the for loop and subsequently the method is $\Omega(n*m)$. This means the complexity of the constructor and methods is $\Omega(n*m + n*m + 1+1+1) = \Omega(2n*m+3) = \Omega(n*m)$ since constants and lower order terms are dropped.

The best-case space complexity for this algorithm is $\Omega(n*m)$, where n is the length of the input array and m is the length of the longest string in the array. In the best-case all strings only have one unique character since those results in the least space created. The constructor first makes 3 variables once, but two of them are arrays which will have a size equal to the input one which makes the complexity $\Omega(n)$ since they are a higher order. Then there is a loop which runs n times and makes a variable i once no matter the input size. It first calls on frequencyCount, which has the same space complexity as the worst case since it only makes three variables no matter the input size and has operations that don't make space, making it $\Omega(1)$. Then it calls on buildHuffman, which makes the same exact space as the worst-case scenario including the findSmallest meaning that that is still $\Omega(1)$, except for the genCode method. The genCode methods in the best case is the same, but its method call to getPath gives a different result. The getPath find the path to the character, but in the best case there is only character, which means that it will only run once and do the string concatenation once to reach the leaf meaning it is $\Omega(1)$. Since it is $\Omega(1)$, that means genCode is also $\Omega(1)$. Since genCode is $\Omega(1)$ that means the buildHuffman method is $\Omega(1)$ since all its operations and method calls are $\Omega(1)$ since the space made by it is constant and is independent of input size. Then there is the method call to encode, which first creates a string variable once no matter the input size. Then it has a loop, with a string concatenation each iteration and the loop iterate m times. Each concatenation is creating one new string each time with one number appended to it, which

means that m iterations results in m string being created. So, in the best-case space complexity of encode is $\Omega(1+m) = \Omega(m)$ since lower order terms are dropped. That means the complexity of the constructor's for loop's body is $\Omega(1+1+m)$, but since there are n iterations of the body because of the loop and it creates a variable i once, the whole loop has a complexity of $\Omega(1+n(2+m)) = \Omega(1+2n+n*m) = \Omega(n*m)$ since constants and lower order terms are dropped. This means the complexity of the constructor is $\Omega(1+n+n*n*m) = \Omega(2n+1+n*m) = \Omega(n*m)$ since constants and lower order terms are dropped. Then there is a method call to `printCodes`, which only creates one inter i , but does so only once no matter the input size so it has a space complexity of $\Omega(1)$ like its worst case. Then there is a method call to `highestCode`, which only creates two variables but those only happen once no matter the input size so it has a space complexity of $\Omega(1)$ just like its worst case. Then there is a method call to `shuffleCodes`, which creates three variables once no matter the input size, which is $\Omega(1)$. In the for loop for that however it creates a new string variable for each iteration and the number of iterations is `encoded.length` which is n , so the space made by that loop is $\Omega(n)$. This means the complexity of the shuffle method is $\Omega(1+n) = \Omega(n)$ since lower order terms are dropped. Then after that is a method call to `printCodeStats`, which is the same as the constructor's for loop except there is a method call to `print Stats` instead of `encode`. The `printStats()` method creates two variables in the for loop itself, but they only happen once no matter the input size, which means the complexity until the print is $\Omega(1)$ since those space is constant no matter input size. Then there is a method call to `encode`, which we determined to be $\Omega(n*m)$, which means the complexity of the method as a is $\Omega(1+n*m) = \Omega(n*m)$ since lower order terms are dropped. This means the complexity of the `printCodeStats` method even with its n iterations and creation of i once is $\Omega(1*n(1+1+m)) = \Omega(n+n*n*m) = \Omega(2n+n*m) = \Omega(n*m)$ since constants and lower order terms are dropped. This means the complexity of the constructor and its four method is $\Omega(n*m + 1+1+1+n*m) = \Omega(2n*m + 3) = \Omega(n*m)$ since constants and lower order terms are dropped.

Problem 4:

The worst-case time complexity for this algorithm is $O(n)$, where n is the length of the input array that is a representation of an encrypted binary tree. This is because in the worst case, this will not take the initial if statement, which returns early, but it still does those comparisons, array retrievals, and a call to the array's `.length` method, which just returns a member data variables, so it is all just a constant time operation of $O(1)$. Then there is another if statement, which is not taken since it would return early, and that just has a comparison in the condition, so it is $O(1)$. After that, there is an integer array

initialization to be equal to the size of the input array using `input.length`. Since it must allocate all elements in it with default values, then it has a time complexity of $O(n)$. After that it updates the new array at index 0 to be 1, which is just a constant time operation meaning it is $O(1)$. After that there are three integer variables being declared and are initialized to 0, which are all constant time operations, so they are just $O(1)$. After that there is a for loop. The loops body only has one if clause with two other if clauses inside of it. The if clause itself has comparisons, and array retrievals, which is constant $O(1)$. Then, it has an update and array retrieval in the first statement, which is a constant time operation of $O(1)$. Following that is updating another variables to be the result of some arithmetic, which is a constant time operations $O(1)$. After that is another if-clause, which has a condition that is just comparisons and array retrials being $O(1)$. Then inside of it has a variable update and arithmetic which are also constant time operations $O(1)$. After that there is another if statement, but it is never taken since it would just return early. So, the time complexity of the if clause after the first arithmetic is $O(1)$. After that is an identical segment which has the same arithmetic comparisons and is clauses but with a different variables. It has an initial update and arithmetic. Then it has an if segment with comparisons, array retrievals, updates, and arithmetic, and an array that is not taken, it is also $O(1)$. That is the entire body of the initial if segment and , which means that that if segment and its condition has a time complexity of $O(1+1+1+1+1+1) = O(6) = O(1)$, since constants and lower order terms are dropped. Since that is inside of the for loop, that means the for loop has a body of $O(1)$. The for loop itself runs from $i=0$ to `input.length`, so it will run n times. Thee for loop itself initializes i once, compares each iteration, and increments each iteration, which means the loop itself has a complexity of $O(2n+2)$, which is $O(n)$, since lower order terms and constants are dropped. Since the complexity of a loop is the complexity of the loop itself time the body, that means this for loop as a complexity of $O(n*1) = O(n)$. So that means the complexity of the algorithm is the sum of those complexities, which is $O(1+1+n+1+1+n) = O(2n+4) = O(n)$, since lower order terms and constants are dropped.

The worst-case space complexity for this algorithm is $O(n)$, where n is the length of the input array. This is because in the worst-case it does not take the first two if clauses, which return early without making any space, and their conditions just do comparisons, array retrievals, and a call to `.length` for an array which does not create any space just returns a value. So, those have a space complexity of $O(1)$. Then after that an array is declared, which has a size equal to n , which means that n spaces are made for the array giving it a space complexity of $O(n)$. After that there are three integer variable declarations, however, no matter how large the input size is, those three variables are only created once,

so their space complexity is $O(1)$. Following that is a for loop, which the loop itself does create a variable i , but no matter how large the inputs size is, it only creates i once so the loop itself has a complexity of $O(1)$. The body of the loop, only contains comparisons, variables updates, call to `.length` for an array, array retrievals, arithmetic and return statements, and if clauses. All these operations do not create any space, in fact there are no variables being declared in the body of the loop only those operations, which are constant in space. So that means the loop's body is $O(1)$. That since the loop does not create any space in any iterations and only creates space once with i being initialized only one time, that means the loop has a space complexity of $O(1)$. That means the space complexity of the algorithm is the sum of those operations, which are $O(1 + n + 1 + 1) = O(n+3) = O(n)$, since constants and lower order terms are dropped.

The best-case time complexity for this algorithm is $\Omega(1)$. This is because in the best case, the algorithm will just make a few comparisons then return. This happens if the input is null, empty, or not a valid encryption, which means the value at index 0 is -1 . In these cases, the algorithm will make three comparisons to check if any one of these are true and if so, to immediately return false. The comparisons also use array retrievals and a call to `.length` of an array, both of which are constants time operations `.length` because it is just returning a member data. Since comparisons, array retrievals, `.length`, and returns are constant in time, that means the complexity would be $\Omega(1+1+1+1+1) = \Omega(5) = \Omega(1)$ since constants and lower order terms are dropped. Even if the input were not null, not empty, and it was a valid encryption, the best case is still $\Omega(1)$. This is because after that initial if clause, there is another one that checks if the target is equal to 1, and if so to return true. This is taken in the best case since it results in the least number of operations avoiding the for loop and array initialization. The reason for that condition is because the root in a valid encryption will always be equal to 1. That means, if this return is taken, the algorithm will have a complexity of $\Omega(1+1+1) = \Omega(3) = \Omega(1)$ since constants are dropped.

The best-case space complexity for this algorithm is $\Omega(1)$. This is because in the best-case the least amount of space is created if the input is null, empty, or not a valid encryption meaning its root, index 0, is -1 . In this case, this is the worst-case, because the algorithm first has these three conditions being checked in an if clause, and if any one of them is met, then the algorithm will just return false. The reason it returns false is because these scenarios would not make it possible to find the target in an encrypted array. This is taken because it is one of the scenarios that results in the least amount of space being created. The if clause has three comparisons, a call to `.length`, and an array retrieval, all of

which do not create any new space with decelerations, `.length` just returns a member data variable meaning it is $\Omega(1+1+1+1+1) = \Omega(5) = \Omega(1)$ since constants are dropped..

Furthermore, that return statement does not declare another variable, which means that in this case the algorithm does not create any space at all which means the space complexity in the best case is $\Omega(1+1) = \Omega(2) = \Omega(1)$ since constants are dropped. Even if the input is not null, empty, or is a valid encryption, the best-case space complexity is still $\Omega(1)$. This is because in this case the first if clause is not taken and it has been established it is just $\Omega(1)$. Then after that there is another if clause that would be taken since it results in the least space being created. This if statement will compare the target to 1, and if it is 1, then it will return true. This if clause just does one comparison to one and has a return statement, which means it does not create any new variables. So that means its space complexity is $\Omega(1+1) = \Omega(2) = \Omega(1)$ since constants are dropped. This if statement happens because if the encryption is valid it will always have a root = 1. So that means the space complexity of this algorithm in the best case is $\Omega(1+1) = \Omega(2) = \Omega(1)$ since constants are dropped, and because the amount of space being created does not change with the input size.

Problem 5:

The worst-case time complexity for this algorithm is $O(n)$, where n is the length of the input array. This is because there is an initial if segment, which is not taken, since it would result in an early return resulting in less operations. The if condition just has a call to an array `.length` method and two comparisons, which the `.length` method just returns a member data variables for a constant time operations, so it has a time complexity of $O(1+1+1) = O(3) = O(1)$ since constants are dropped. Then after that a Boolean variable is declared and initialized, but it only happens once regardless of input size, so it has a time complexity of $O(1)$. Then after that is a print statement, and we are treating print statements as constant time operations. So that has a complexity of $O(1)$. Then after that is a for loop. The for loop has a body, which first declares and initializes a string variable to be the string at a certain index of the input, which is an array retrieval. Since initializations and array retrievals are constant time operations, this has a complexity of $O(1+1) = O(2) = O(1)$ since constants are dropped. Then after that is an if segment, the condition of that segment first calls on `.length()` for a string, which is a constant time operation since it is returning a variable that is kept track of in the class, and then does a comparison. Since these are constant time operations, the condition itself is $O(1)$. The if segment is also taken at every iteration in the worst case because it results in most operations being done, otherwise no other operations would occur in the loop. Then the body has an if statement,

which is only taken if first is false, which is only after the first iteration of the loop. The if segment's body just has a print statement which is $O(1)$. Then after that are three more print statements all which are constant time, meaning they have a complexity of $O(1)$. Then after that, first is updated to false, which is just a constant time operation $O(1)$. That means the complexity of the entire if segment is $O(1+1+1+1) = O(4) = O(1)$ since constants are dropped. The internal if segment being taken does not matter so much because it is just constant time like the rest of the body, so of the body so it will not affect the time complexity. So, the complexity of the for loop's body is $O(1+1) = O(2) = O(1)$ since constants and lower order terms are dropped. The for loop itself initializes $i=0$ once, and compares i to `input.length`, and increments i , but it does the comparison and increments each iteration until that condition is not met which means it does n iterations. This means the loop itself has a complexity of $O(2n+2) = O(n)$ since constants and lower order terms are dropped. After the loop there is another print statement which is $O(1)$. So, the overall time complexity of this algorithm is $O(1+1+1+n+1) = O(n+4) = O(n)$ since lower order terms are dropped.

The worst-case space complexity for this algorithm is $O(n)$, where n is the length of the input array. This is because initially there is an if segment, which does not get taken since it results in less space being made. The condition though still does happen and it is just comparisons and a call to an array's `.length` method, which does not make any variables just returns one, so the condition is constant in space being $O(1)$. Then after that is a boolean variable being declared, but no matter how large the input size grows, that variable will always be declared once, making it $O(1)$. Then, after that is a print statement, which we are treating as constant, making it $O(1)$. Following that is a for loop. The for loop's body first creates a string variable and initializes it to be the corresponding value from the input at a certain index, so it is $O(1)$. After that is a large if segment, which its condition and body only contain print statements, comparisons, updates, and a call to `strings.length()` method, which just returns a variable not create one. So, the entire if segment there does not create any space giving it a complexity of $O(1)$. So, the body of the for loop is $O(1)$, since the string variable is the only thing being created in the body. The loop itself, creates a variable i once no matter how much the input size is so its space complexity is $O(1)$. The loop itself runs n times since it starts at $i=0$ and increments it until i is not less than n . Since the loop runs n times, then that means the string variable in the body is being created n times. So the loop itself has a space complexity of $O(1+n) = O(n)$ since lower order terms are dropped. After the for loop, there is another print statement, which is constant. This means the complexity of the algorithm is $O(1+1+1+n+1) = O(n+4) = O(n)$ since lower order terms and constants are dropped.

The best-case time complexity for this algorithm is $\Omega(1)$. This is because the first if segment would be taken since it results in the least operations. This is because in the best-case scenario the input is 0 or the max length variable is less than or equal to 0, this would result in the first if statement being taken. The reason that 0 is chosen is because it is impossible to have a string with a length less than 0. The first if statement has the conditions that if the length == 0 or max <= 0, then it would be taken. Its body also just has a print statement and a return statement. Since we are treating printing as constant in time, that means the body of the if segment has a complexity of $\Omega(1+1) = \Omega(2) = \Omega(1)$ since constants are dropped. Then the if condition itself has two comparisons and a call to .length for an array, which just returns a member data variable, so that means it has a complexity of $\Omega(1+1+1) = \Omega(3) = \Omega(1)$ since constants are dropped. That means the if segment has a complexity of $\Omega(1+1) = \Omega(2) = \Omega(1)$, since constants are dropped. Thus, in the best-case, the algorithm has a complexity of $\Omega(1)$.

The best-case space complexity for this algorithm is $\Omega(1)$. This is because in the best-case scenario the first if segment would be taken, since it results in the least amount of space being created. The first if segment has the condition that the length of the array is 0 or the target integer to find strings with a length less than it is less than or equal to 0. The reason that is the condition is because it would be impossible for a string to have a length less than 0. So, that means the best-case scenario would be a input of length 0 or a max length that is less than or equal to 0. The if statement's body just has a print statement and a return statement. Since we are treating print statements as constant in space, and a return statement does not create any new variables. That means the space complexity of the body is $\Omega(1+1) = \Omega(2) = \Omega(1)$ since constants and lower order terms are dropped. The condition also just has two comparisons, and a method call to string's .length() method, which just returns a value and does not create any new space, neither do comparisons. This means that the space complexity of the condition is $\Omega(1)$ since no space is being created. That means the complexity of the if segment is $\Omega(1+1) = \Omega(2) = \Omega(1)$ since constants are dropped. Thus, in the best case scenario, the time complexity is $\Omega(1)$, since no matter the input size the amount of spacing being created will not change.

Problem 6:

The worst-case time complexity for this algorithm is $O(n \log n)$, where n is the number of nodes in the BRTree. This is because the inRange() method first creates and

initializes a linked list of integers. This itself has a time complexity of $O(1)$. Then after that it calls on the `addKeysInRange()` method, which will take that linked list, the root, and the range of numbers to find all nodes that are in the range and add it to the linked list. That method first has an if statement that will return and exit the method if the node is null. Then after that it will enter an if statement, and check if the node is in the range, inclusive. This is done with two comparisons and two calls to `node.getData()`, which just returns that data member variable of the node, which is constant. So, the if segment has a complexity of $O(1)$, and its body will add to the front of the linked list, the node's data, which does create a new node but that is constant in time since it just updates variables. Adding to the front of the linked list is also constant in time since it just updates pointers. This means the if segment has a complexity of $O(1+1+1+1+1) = O(5) = O(1)$ since constants and lower order terms are dropped. After that is a recursive call to the method but with the node's left child, and after that is another recursive call but on the node's right child. This means that the method does a preorder traversal. This means that it will visit every single node in the tree and do the comparisons and add it to the linked list because that results in the most operations. So, that means all of the comparisons and adding, which have a combined complexity of $O(1+1) = O(2) = O(1)$ since constants are dropped. And since this is doing an inorder traversal, that means these will be capped n times, meaning that for the `inRange()` method, the call to `addKeysInRange()` has a time complexity of $O(n)$, since it will do the $O(1)$ at all nodes. After that is a while loop, which runs until the linked list is empty, which means it runs n times in the worst case, since all nodes will be in range in the worst case. The body of the while loop calls on the `remove()` method I made, and the remove method of a linked list for the linked list. The remove method of a linked list is constant in time because it is just updating pointers, meaning it is $O(1)$. The remove method I made on the other hand is not. This method will first initialize a variable to be the result of the search method on the root. The initialization itself is $O(1)$ since it's a constant time operation, but the search method is not. The search method will do a search through the binary tree and check the node to see if it is null or equal to the method. This means it does comparisons and calls to `getData()`, which are constant time operations being $O(1)$. If the node's value is less than the target it will do a recursive call on the left subtree, and if greater than it will do a recursive call on the right subtree. Since this will only ever check one level of the tree, that means the number of searches will be equal to the height of a tree, and since this tree is a `RBTree`, that means its height will be $\log n$ since it is self-balancing. That means the search method has a complexity of $O(\log n)$ since those $O(1)$ comparisons will happen $\log n$ times. After that is an if statement, which is not taken because it results in less operations. The if statement's condition is just a comparison making it $O(1)$. Then there is an if else segment, where the if part is taken only if the node has two children and the else node is taken otherwise. The body of both are nearly identical, with the only differences being

that for the if body, there is a while loop and other updates, and the else body has another if segment inside of it. Both have an if segment, where they get the color of the node with the `getColor()` method and a comparison. That method is constant since it is just returning a variable so it is the comparison so the if condition for both is just $O(1)$. Then in the body of it is a method call to `removalPrep()`, this is taken in the worst case since it results in the most operations. That method has 8 sections, the first one just has two comparisons and calls to `getColor` and `getParent`, both of which just return a value and thus constant time. So it has a complexity of $O(1)$, and it is not taken since it results in the least operations. After that is the second one, which initializes two variables and uses the `getParent` and `getSibling` method, the `getSibling` method is constant time since it just calls on `getParent`, `getLeft`, `getRight`, and does comparisons and returning all of which are constant time operations. So that means that so far case 2 is $O(1)$. After that it has an if segment, with its conditions two comparisons and a call to `getColor`, so that means it is $O(1)$. Then its body calls on `set red` and `set balck` both of which are constant time, then there is an if statement with a comparison and both bodies for it and the else just call a `rotate` method both left and right have the same complexity since they are just mirrored implementations of each other. Both rotates have calls to `getRight`, `getLeft`, `getParent`, `setLeft`, `setRight`, `setParent`, and other updates, all of them only happen once in the method, which means that both rotation methods have a time complexity of $O(1)$. Then that means case 2 has an overall time complexity of $O(1+1+1+1+1+1) = O(6) = O(1)$, since constants are dropped. Then case 3 first updates two variables using the `getParent` and `sibling` method, then has an if condition that does one comparison, then another one that has 5 comparisons, and methods calls to `getLeft`, `getRight`, and `getColor`, which are all constant time operations, and its body, has a call to `setRed`, a recursive call to this method, but on this node's parent, and then returns. The time complexity of all this excluding the recursive call for now is $O(1)$, since it is just constant time operations. In the worst case, scenario case three could be taken each iteration of the recursive call, which means that it would travel on the path from the node to the root, meaning that it will travel up the height of the tree being $\log n$ since it's a self balancing tree. That means the complexity of case 3 in the worst-case is $O(1 + \log n) = O(\log n)$ since lower order terms are dropped. Then case 4 also has the same 2 initial updates as case 3, which is $O(1)$. Then it has an if segment with a comparison, and another one in its body, which has 5 comparisons and calls to `getLeft`, `getRight`, and `getColor`. That body is never taken in the worst case since it has a return statement, which would result in it leaving the method early resulting in less operations happening. So that means the complexity of case 4 in the worst case is $O(1)$. Then case 5 has the two same updates and method calls initially as the previous one, and then has an if segment with one comparison, and then another if segment inside of it with 5 comparisons and calls to `getLeft`, `getRight`, `getColor`. Then in its body, then it has calls to `getSibling`, `getLeft`, and

setRed and Set Black, and then rotateRight. All of these are previously established as constant time operations, which means that case 5 is only made up of constant time operations making it $O(1)$. Case 6 has the exact same method calls and if segment structure as case 5, with the only difference being the body of the inner if segment has a call to rotateLeft and getRight instead of getLeft and rotateRight, but either way they are constant time operations as well. This means that case 6 has an overall time complexity of $O(1)$. Step 7 has the same two updates and method calls as the previous ones, but then it has an if segment with one comparison, followed by an if else segment with one comparison, but the body of both segment just have a call to setRed or setBlack, both of them are $O(1)$, then at the end it sets the parent to black with setBlack(). That means the overall complexity of this step is $O(1)$. Finally there is step 8, which has an if segment with one comparison, another if-else segment but which path it takes does not matter since both do the same thing with an getLeft or getRight method call, a comparison, a setBlack, and a call to rotate left or right depending on the segment. Either way that means that the complexity of these are both $O(1)$ since they are constant time operations. This means that case 8 has an overall time complexity of $O(1)$. That means the overall time complexity of the removalPrep method is $O(1+1+\log n+1+1+1+1+1) = O(\log n + 7) = O(\log n)$ since constants and lower order terms are dropped. That brings us back to the remove method, and both bodies in the if-else segment also call on BST removal, which only has comparisons, method calls to getLeft, getRight, setLeft, setRight, setParent, getParent, and updates none of them happen more time with input size and they are all constant time operations, which means that BST removal has a time complexity of $O(1)$. Then the else segment has one more if segment, which has 2 comparisons, a getColor method call, and a setBlack method call, all of which are $O(1)$, that means that if segment of it is $O(1)$. So the complexity of the else segment is $O(\log n+1+1) = O(\log n + 2) = O(\log n)$, since constants and lower order terms are dropped. The if segment has a body that first initializes a node to be the node's left child with getLeft, which means it is $O(1)$. Then a while loop to find the predecessor of the node, which will go until the current node no longer has a right child, thus being a predecessor. The while loop's condition is a comparison and a method call to getRight(), which means it is $O(1)$ since they are all constant time operations. Its body updates predecessor to be its right child using getRight, which means that the body is $O(1)$ since they are constant time operations. Since the loop runs until a right child in the left subtree is not found, then that means in the worst case it may have to travel down the height of the tree, which is $O(\log n)$, since it is a self balanced tree height will be $\log n$ not n . Since the complexity of a loop is the loop itself time the body, that means the while loop has a complexity of $O(\log n * 1)$, which is $O(\log n)$. Then after that is an update with a method call to getData() and after that is the removalPrep which we have already evaluated and bstRemoval. After that it just calls setData() which is constant in time. So

the complexity for the if segment as a whole is $O(1 + \log n + 1 + \log n + 1 + 1) = O(2 \log n + 4) = O(\log n)$ since constants and lower order terms are dropped. Since both the if and else are $O(\log n)$, that means the removal method's if-else segment has a complexity of $O(\log n)$. That means the complexity of the remove() method is $O(\log n + 1 + 1 + \log n) = O(2 \log n + 2) = O(\log n)$ since lower order terms and constants are dropped. So going back to the inRange() method, its while loop calls our remove method and the linked lists remove method in its body, meaning it has a complexity of $O(\log n + 1) = O(\log n)$ since power order terms are dropped. The while loop has a complexity of $O(n)$ itself since it will continue until the list is empty, using the isEmpty method which just does a comparison and a return both constant time operations, which is n calls in the worst case. The body $O(\log n)$, which means it is $O(n * \log n)$ since the complexity of a loop is the complexity of the loop itself times the body. That means the complexity of the inRange() algorithm is $O(1 + n + n \log n) = O(n \log n)$ since lower order terms are dropped and both 1 and n are lower order than $n \log n$.

The worst-case space complexity for this algorithm is $O(n \log n)$, where n is the number of nodes in the RB tree. This is because the inRange() method first just creates an empty linked list, which only happens once in the algorithm so it has a space complexity of $O(1)$. Then after that it makes a call to addKeysInRange(), which does an in order traversal of the tree to do the following operations: return null if the node is null, call getData() to get the node's value, does 2 comparisons to see if it is in the target range, if so then add it to the linked list. Adding to the linked list is done using the addFirst() method of the linked list, which creates a new node with that value and updates pointers. Since it creates new space with the node, it has a space complexity of $O(1)$. In the worst case scenario, all nodes will fall in range meaning that all nodes will be added to the linked list, which means that the space complexity of this method in the worst case is $O(n * 1) = O(n)$. After that is a while loop. Inside of that while loop is a call to remove() for a linked list, and remove() which is a method I made. Remove() for a linked list just updates pointers and returns a value, which does not create any space, so its complexity is constant $O(1)$. The remove() method I made, first creates a new BRNode variable and calls the search method. The BRNode variable declaration is $O(1)$, since it will only happen once in remove no matter how large the input size is. The search method only has comparisons, returns, calls to getData, and recursive calls. Since comparisons, returns, and calls to getData do not create any new space, that means the method, excluding the recursive calls for now, does not create any space. That means its recursive calls don't either, and since this will search through the height of the tree being $\log n$ since it's a self-balancing tree. This means that even in the worst-case this method has a space complexity of $O(1)$. That means the first statement in the remove method has a space complexity of $O(1)$ since the new node

created is the only new space made. After that is a if segment with a comparison in the condition, but this is not taken since the body returns early, and the comparison does not create any variables meaning its space complexity is constant $O(1)$. After that an integer variable is created, but since it only happens once in the remove method no matter the input size it's complexity is $O(1)$. Then there is an if else segment, but both segments have similar method calls. The first one is BST remove, which appears in both and that method does create variables. The bstRemove method only has three if segments, each one has comparisons, calls to getParent, getRight, getLeft, setLeft, setRight, updates, returns, and setParent. All these method and statements are constant in space since they do not create any variables and just update them. The first two if segments do not create any variables and just do those methods. The third if segment creates a variable of BRNode if taken. In the worst case this means that the BSTRemove method does create a variables and has a complexity of $O(1)$. The other things both if-else segments have in the remove method is a comparison, call to get color, and removalPrep. Comparisons and getColor do not create any variables so they have a constant space complexity $O(1)$. The removal prep method, on the other hand, does create them. This method has 8 sections, the first, fourth, and seventh sections do not create any variables at all and just have comparisons and calls to getColor, setBlack, setRed, getLeft, getRight, getParent, which do not create any space so their space complexity is $O(1)$. Section 2 creates two variables but only happens once in the method no matter the input size so its complexity is $O(2) = O(1)$ since constants are dropped. Case 3 has the same comparison and calls to the get methods like the previous ones, but also has a recursive call to this method followed by a return statement. In the worst case scenario this may do a recursive call each time and since it is done on the parent this means in the worst case it will happen $\log n$ times, not n since the tree is self-balancing. Furthermore, the two nodes created for section 2 will happen no matter what, which means that that space $O(1)$ will be created $\log n$ times if in the worst case. This means that section three has a space complexity of $O(\log n)$. Then sections 5 and 6, are the same as each other but have mirrored get and rotate method. Both start with updates followed by method calls to getSibling and getParent both of which don't create space. Then in both, there is an if segment with comparisons, calls to getLeft, getRight, and getColor, which also don't create any space. Then in both bodies, there is a call to setRed, setBlack, and getLeft or getRight, but all these do not create any space, meaning so far the complexity of both sections is $O(1)$. Then in both there is a call to a rotate method, right in 5 and left in 6, both rotation methods are the exact same but mirrored in which left or right child is being updated. Both rotation methods create two BRnode variables at the start, which means its space complexity so far is $O(2) = O(1)$ since constants are dropped. Those two variables there are only created once in their method no matter how large the input size is, and even then both methods only have if-segments, comparisons, call to the

get and set method, all of which do not create any space. This means the space complexity of the rotates are $O(1)$ since they do create space. That means sections 5 and 6 do make space and thus have a complexity of $O(1)$. Finally, there is section 8, which just has updates, call to setBlack, getLeft or right, and rotate left or right. The updates and set and gets do not create space, but the rotates do, which means that the space complexity of this section is $*1$. This means that the space complexity of this method is $O(1+1+\log n+1+1+1+1+1) = O(\log n + 7) = O(\log n)$, since constants and lower order terms are dropped. So in the removal method, the else clause has an if segment, but it only has comparisons, and calls to getColor and set black, which don't create space. This means the body of the else segment has a space complexity of $O(1 + \log n + 1) = O(\log n + 2) = O(\log n)$ since constants and lower order terms are dropped. Then the body of the if segment creates a node variable once, which is $O(1)$ and then has a while loop, but that loop only does a comparison, call to getRight, and update, which do not create any space, which means that while loop does not make any space and is just $O(1)$. Then it call on setData, which does not create any variables just updates, then so its space complexity is $O(1)$. This means the complexity of the if body is $O(1+1+\log n+1+1) = O(\log n + 4) = O(\log n)$ since constants and lower order terms are dropped. Since the body of both the if and else segment are $O(\log n)$ that means as a whole it is just $O(\log N)$ regardless of the route. This means the space complexity of the remove method is $O(1 + 1 + \log n) = O(\log n + 2) = O(\log n)$ since constants and lower order terms are dropped. That means the body of the while loop in the inRange() method is $O(\log n + 1) = O(\log n)$ since lower order terms are dropped. The while loop itself, will run until the linked list is empty by checking the isEmpty method, which just does a comparison and return, which do not create variables. That means it is $O(1)$, but the while loop runs until the list is empty and removes an element each iteration meaning it will run n times. That means the remove() method I made will be called n times, thus the space being created by the while loop is $O(n * \log n) = O(n \log n)$. That means the space complexity of the inRange() method is $O(1+n+n \log n) = O(n \log n)$ since lower order terms are dropped.

The best-case time complexity for this algorithm is $\Omega(n)$, where n is the number of nodes in the tree. This is because in the best-case scenario, no nodes in the tree are in the range. In this case, the inRange method initializes a linked list once, which means it is $\Omega(1)$. Then it calls on the addKeysInRange() method, which first does a comparison for an if segment, which is $\Omega(1)$ since comparisons are constant time operations. Then after that are two comparisons and call to getData, both of which just return variables and are constant time operations, meaning it is $\Omega(1)$. The body of the if-segment is never taken otherwise it would result in more operations. Then following are two recursive calls on the

left and right child, which means this is doing an inorder traversal. So far, the complexity of the method is $\Omega(1)$. Since it is doing an inorder traversal that means it is doing those $\Omega(1)$ operations n time making the complexity of this method $\Omega(n)$. Then after that, in the `inRange()` method, where is a while loop, which will only run if the linked list is not empty, it is checked using the `isEmpty` method of a linked list, which just does comparisons and return, which is $\Omega(1)$. The while loop is never taken in the best case since it would result in less operations, and the linked list will only have elements that are in the range and the best case is nothing in range. This means the complexity of this method in the best case is $\Omega(1+n+1) = \Omega(n+2) = \Omega(n)$ since constants and lower order terms are dropped.

The best-case space complexity for this algorithm is $\Omega(1)$. This is because in the best case the amount of space being created by the algorithm does not change based on the input size. In the base-case, no matter how many nodes there are, as long as none of them are in the target range, then it is the best-case. The `inRange()` method first creates a linked list variable but it is empty, and will only happen once no matter the input size, so it has a complexity for $\Omega(1)$. Then it has a method call to `addKeysInRange()`, which first has a if statement if the node is null, but this and its body do not declare any variables and don't create space. Then there is an if statement with comparisons, and calls to `getData()`, but those do not create any variables or space. This means that so far, there is no space being created by this method. That if segment's body is not taken since it results in more space being created and only happens if the node is in range, but in the best case, no nodes are in range thus no nodes are added to the list. Then there are two recursive calls to the method, but since the method does not create any space in the best case, no matter how many recursive calls are made, the space made by the algorithm is 0 and the space complexity of this method is $\Omega(1)$. Following that, in the `inRange()` method, there is a while loop, which calls on `isEmpty()`, which does not create space and just does comparisons and returns, meaning it is $\Omega(1)$. Since nothing is added to the linked list in the best case that means the while loop is empty and is not taken, thus being $\Omega(1)$. That means in the base case space complexity of this is $\Omega(1+1+1) = \Omega(3) = \Omega(1)$, since constants are dropped, and since no matter how many nodes in the tree there are, in the best case the amount of space being made does not change.